

From Suspicious Patterns to Reportable Findings: LLM-Assisted Static Security Review of Android Automotive IVI APKs

Hoyeon Kim

Department of Mobility Systems Engineering
Yonsei University, Seoul, Republic of Korea
cabinkhy@yonsei.ac.kr

Abstract—Android Automotive OS (AAOS) in-vehicle infotainment (IVI) applications combine ordinary Android application surfaces with OEM-controlled data, services, permissions, and trust boundaries. In this setting, a suspicious code pattern is not enough to determine whether a finding is reportable, because reportability depends on Android platform semantics such as component export state, caller reachability, permission gates, protected broadcasts, and route binding. We do not claim APK-wide vulnerability discovery. Instead, this paper studies whether LLM-generated Android/AAOS security claims can be made auditable and substantially less noisy when reportability is gated by Android context evidence and conservative role-threshold adjudication. The workflow uses deterministic APK triage to prioritize security-relevant code regions, LLM-assisted candidate generation to formulate reviewable claims, Android context validation to test whether those claims survive platform constraints, and role-threshold adjudication to make final reportability decisions. We evaluate the workflow as candidate-level adjudication on a 47-candidate reportability corpus drawn from PleOS-customized target APKs and external Android positive and negative controls. Treating the broad LLM-generated candidate pool as a report-all candidate baseline yields 38 reference-positive and 9 reference-negative candidates. After Android context validation and the selected two-of-three role-threshold rule, the workflow reports 37 findings with zero false positives observed among the 9 reference-negative candidates in this 47-candidate corpus and one low-severity false negative against the 38 reference-positive candidates. A paired McNemar exact test gives $p = 0.0215$ on this corpus and is interpreted as exploratory paired evidence, not a confirmatory generalization claim. The result supports a bounded methodology claim: LLMs are useful for generating and organizing reviewable security claims, but AAOS/IVI reportability requires Android context evidence and conservative claim boundaries.

Index Terms—Android Automotive OS, in-vehicle infotainment, static analysis, large language models, security claim adjudication, reportability assessment, Android context validation, mobile application security, false-positive reduction, threat analysis and risk assessment.

I. INTRODUCTION

Android Automotive OS and PleOS-style IVI applications combine ordinary Android application surfaces with IVI-local data, vehicle-service-adjacent permissions, local content providers, Binder services, route state, OEM trust boundaries, and security-sensitive configuration. In this paper,

vehicle-adjacent means data, services, permissions, and OEM-controlled boundaries that are close to vehicle systems but do not by themselves imply steering, braking, powertrain, or autonomous-driving control. In this environment, a suspicious API call or decompiled code pattern is not enough to decide whether a finding should be reported. Android exploitability depends on context: whether the component is exported, whether a normal same-device caller can reach it, whether signature or privileged permissions block it, whether route binding keeps data internal, whether a broadcast is protected, and whether framework controls prevent caller influence. Route binding means that the relevant data or control path is fixed to an internal package, component, account, or service route rather than influenced by an untrusted caller.

A reportable finding is a candidate whose static evidence supports a reachable security boundary violation under the stated same-device threat model, even if runtime exploitability or vehicle-level impact is not fully demonstrated. A claim is a reviewable security hypothesis that names an entrypoint, a source/sink behavior, an Android context condition, and a potential security consequence. This paper studies a reportability-centered workflow for LLM-assisted static security review of AAOS/IVI APKs. The central claim is deliberately bounded: the LLM proposes reviewable security claims after deterministic triage, but reportability is decided only after Android context validation and role-threshold adjudication. In short, the LLM helps create reviewable claims; Android semantics decide whether those claims remain reportable.

The key observation is that LLMs are useful at turning prioritized decompiled-code regions into candidate security hypotheses, but reportability requires Android-domain evidence. A provider exposure claim, for example, must survive checks for export state, permission gates, caller assumptions, and sensitivity of returned rows. A receiver claim must survive protected-broadcast semantics. A permission-management claim must survive caller-route and package-name control checks.

The key distinction is between a suspicious pattern and a reportable claim. A manifest-only scan may flag an exported provider, but the claim should be rejected if the provider is guarded by a signature permission or returns no caller-controlled sensitive rows. Conversely, a provider with no permission gate and IVI-local sensitive rows remains reportable

under the same-device threat model even without a remote exploit. This difference motivates a reportability workflow: the unit of review is not a raw pattern, but a claim that must survive Android context checks.

This paper makes the following contributions:

- 1) It proposes a reportability-centered LLM-assisted static security review workflow for AAOS/IVI APKs.
- 2) It defines a candidate-finding unit and project-derived reference-label protocol for a 47-candidate adjudication corpus.
- 3) It shows that Android context validation and role-threshold adjudication make rejection rationales auditable, reduce observed false positives from 9 to 0 relative to a report-all candidate baseline on this corpus, and preserve 37 of 38 reference-positive candidates.
- 4) It maps final reported findings to AAOS/MASVS/TARA-oriented reporting artifacts while maintaining a public/private evidence boundary.

II. BACKGROUND AND RELATED WORK

A. Android Static and Taint Analysis

Android application security has a long line of static analysis work on lifecycle-aware taint tracking, inter-component communication, component hijacking, and information-flow analysis. FlowDroid models Android lifecycle and callback behavior for precise taint analysis and introduced DroidBench as an Android taint-analysis benchmark, while Amandroid and IccTA emphasize inter-component data-flow and privacy-leak analysis across Android components [1]–[3]. DroidSafe builds a deeper Android runtime and API model for information-flow analysis, and CHEX focuses on component hijacking vulnerabilities such as permission leakage, unauthorized data access, and intent spoofing [4], [5]. ReproDroid-style comparative work shows that Android taint-analysis precision and coverage depend heavily on modeling assumptions, source/sink definitions, and reproducible tool configuration [6].

B. Android ICC, Component Exposure, and Exploitability

The closest Android security line is inter-application and inter-component communication analysis. ComDroid characterized Android message-passing risks and showed that intents and exported components create an application attack surface [7]. Epicc formalized scalable inter-component communication mapping and reduced false positives in ICC discovery by extracting more complete intent specifications [8]. RAICC later showed that models centered on documented framework ICC methods can miss atypical ICC links, reinforcing that ICC modeling completeness and reportability review are related but distinct problems [9]. SEALANT combines static inference of vulnerable communication channels with runtime monitoring, while Android Wicked Delegation work asks whether privileged actions depend on attacker-controlled inter-app data [10], [11]. LetterBomb moves further toward exploit generation by synthesizing ICC inputs for Android vulnerabilities [12].

These systems primarily discover, model, monitor, or exploit Android communication surfaces. Our workflow uses a different unit: a candidate security claim. The goal is not to replace

ICC or taint analysis, but to adjudicate whether LLM-proposed claims remain reportable after Android platform semantics—export state, permission gates, protected broadcasts, route binding, caller reachability, and AAOS trust boundaries—are applied.

C. Context-Dependent Android Exploitability

Android component risk is context-dependent. Exported activities, services, receivers, and providers may be benign when protected by permissions, explicit package binding, framework checks, or protected-broadcast semantics, but security-relevant when reachable by an unprivileged same-device caller. Android platform documentation describes how `android:exported`, Android 12 exported-component requirements, permission-based access control, signature-level permissions, provider `readPermission/writePermission` declarations, and receiver exposure shape whether another app can invoke a component or reach sensitive functionality [13]–[18]. AOSP framework sources also provide the first-party basis for platform-protected broadcasts such as `BOOT_COMPLETED` [19]. These sources motivate this paper’s context-validation layer: a suspicious API or manifest entry is not itself a reportable finding unless the Android platform context preserves a reachable security boundary.

D. Android Automotive and IVI Security

AAOS and IVI deployments add vehicle-adjacent trust boundaries to Android application security. Android for Cars and AAOS documentation distinguish vehicle-installed Android applications from phone-projected experiences, while AOSP automotive documentation describes Vehicle HAL abstractions and vehicle-system isolation as boundaries between Android framework code and safety-critical subsystems [20]–[23]. Prior IVI and automotive Android security work has studied IVI ecosystems, Android Auto and OBD-II applications, and AAOS architecture or access-control gaps [24]–[27]. This paper is narrower: it treats vehicle-adjacent APIs as context that must constrain the claim boundary, and it focuses on IVI APK static review plus selected same-device emulator evidence rather than actuation claims.

E. LLM-Assisted Security and Alert Adjudication

Recent LLM-assisted security work explores vulnerability detection, static-analysis augmentation, repository-level reasoning, alert triage, and structured prompting. IRIS is a close comparison point because it combines LLM reasoning with static analysis for repository-level vulnerability detection, including LLM-assisted specification inference and contextual analysis [28]. Other work studies LLMs as aids for static analysis or alert adjudication, including bug detection, false-positive filtering, and alert triage, while surveys organize the broader LLM-for-security field by problem formulation, dataset design, and evaluation practice [29]–[32]. Recent LLM vulnerability-detection benchmarks also emphasize that vulnerability detection remains difficult across languages and that

context-poor or function-only datasets can inflate apparent capability [33]. This paper positions the LLM as a claim-generation and review component rather than as the final reportability authority. The differentiating point is the explicit Android/AAOS context evidence package and role-threshold reportability gate. The evaluation therefore asks whether LLM-proposed claims can be made auditable under Android platform semantics, not whether a general-purpose model can replace a static analyzer or a human security reviewer.

F. Mobile and Automotive Security Standards

OWASP MASVS, MASTG, MASWE, and MASTG tests for IPC exposure and Android permissions provide public mobile-security vocabulary for reverse engineering, component exposure, and permission review [34]–[38]. Automotive cybersecurity work uses threat analysis and risk assessment language associated with UNECE R155, ISO/SAE 21434, HEAVENS, and EVITA [39]–[43]. In this paper, AAOS/MASVS/TARA mapping is a reporting vocabulary and prioritization frame, not proof of vehicle-level risk.

Taken together, existing Android analyzers primarily optimize program-analysis coverage, ICC modeling, exploit generation, or predefined vulnerability classes, while LLM-assisted systems often evaluate whether models can detect or adjudicate vulnerabilities from code context. Neither line directly addresses the reportability problem faced in decompiled AAOS/IVI APK review: a candidate must be expressed as a claim and then rejected or retained according to Android reachability, permission, broadcast, and OEM trust-boundary evidence. This work studies that reportability workflow under a bounded same-device threat model.

III. THREAT MODEL AND SCOPE

We consider a same-device unprivileged Android application as the primary attacker. This model represents a normal Android app that is sideloaded, installed through an app-market path, or installed in the emulator test environment without system privileges. The attacker may send ordinary intents, query exported providers, interact with exported components, or exercise app-level surfaces permitted by the Android platform. The model is intentionally weaker than a system compromise and stronger than a purely remote network attacker. We do not assume root access, system UID, privileged permissions, OEM signing keys, undisclosed credentials, or physical vehicle access unless a case explicitly states it.

The evaluated results are candidate-level and corpus-bound; they do not estimate APK-wide discovery recall or AAOS ecosystem prevalence.

IV. OVERVIEW OF THE REPORTABILITY REVIEW WORKFLOW

Figure 1 summarizes the workflow. The pipeline begins with APK collection and decompilation, narrows decompiled code using deterministic triage, asks an LLM to generate reviewable candidate claims, validates those claims against Android context, and then applies role-threshold adjudication. The workflow is best read as artifact transformation:

prioritized code regions become candidate claims, candidate claims become evidence packages, evidence packages become report/reject decisions, and retained decisions become public-safe reporting artifacts.

A. Deterministic APK Triage

Deterministic triage narrows the decompiled APK to security-relevant regions. Its input is the JADX output plus manifest and resource artifacts. A heuristic obfuscation screen records identifier-name entropy and rename plausibility as a review-planning signal, not as vulnerability severity, reportability, or final finding quality. The triage first records keyword and manifest hits for exported components, providers, broadcasts, WebView use, permission APIs, cleartext networking, hardcoded credentials, cryptographic APIs, dynamic loading, and IVI-specific service surfaces. Those hits are grouped into security-category queues and converted into short code/evidence snippets for review. The output is therefore not a vulnerability decision; it is a prioritized set of component, method, configuration, and manifest breadcrumbs that constrain what the LLM sees.

B. LLM Candidate Claim Generation

The LLM-assisted candidate-generation step turns prioritized code locations into candidate security claims. The prompt input contains the category, APK/component identifier, public-safe code or manifest snippet, surrounding call or configuration context when available, and the requested evidence format. The expected output is not a free-form vulnerability declaration; it is a claim record containing the suspected entrypoint, source/sink behavior, category, evidence citation, tentative security consequence, and explicit uncertainty or missing context. A candidate claim is a reviewable statement such as “this exported provider may expose sensitive rows,” “this permission-management call may be reachable from an untrusted caller,” or “this WebView path may cross a trust boundary.” Claims without evidence breadcrumbs or claims that exceed the supplied code/context are treated as weak evidence and must survive later validation before reporting. Proprietary snippets, prompt rows, raw logs, and unredacted runtime evidence remain outside public artifacts.

C. Android Context Validation

Android context validation checks whether the candidate claim survives platform evidence. It examines manifest exposure, caller reachability, permission and signature gates, route binding, protected broadcasts, explicit component/package binding, framework-mediated controls, and trust-boundary crossing. Its output is a survival or blocking rationale attached to the evidence package; it does not by itself create a new metric row or claim to be a complete APK-wide reachability analyzer. This is the point at which official Android semantics enter the workflow: exported components and providers shape same-device reachability, permissions and signature gates constrain callers, and protected broadcasts prevent ordinary apps from spoofing selected framework events [13], [15]–[17], [19].

TABLE I
THREAT MODEL AND EVIDENCE BOUNDARY

Item	In Scope	Boundary
Attacker	Same-device unprivileged Android app.	No root, system UID, privileged permission, OEM signing key, or undisclosed credential.
Target surfaces	Exported components, providers, receivers, services, WebView paths, credentials, logs, network configuration, and IVI-local data flows.	Not an exhaustive proof of every Android call path or every app.
Evidence level	Static APK evidence plus selected same-device emulator validation.	Runtime traces refine claim strength; they do not change the static metric.
Non-goals	Public-safe reporting, mapping, and bounded emulator PoC evidence.	No physical vehicle actuation, steering, braking, gear, powertrain, or autonomous-driving control claim.

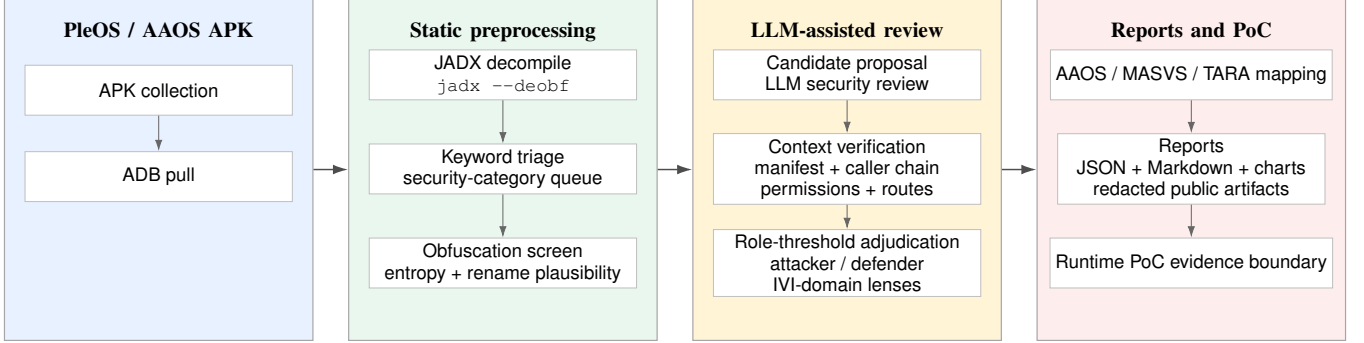


Fig. 1. Reportability-centered LLM-assisted static security review workflow, adapted from the README pipeline. The LLM generates reviewable claims; Android context validation and role-threshold adjudication decide reportability.

D. Role-Threshold Adjudication and Final Reporting

The final reporting gate uses three role-conditioned review lenses: attacker, defender, and IVI-domain. Each lens receives the same evidence package and emits a report/not-report decision with rationale. Severity and case priority are available as context, but the threshold is based on reportability decisions rather than severity voting. A candidate is reported by the selected rule when at least two role lenses classify the claim as reportable. The two-of-three threshold is the selected archive rule because, on this corpus, it removes the observed false positives while avoiding the larger recall loss of the three-of-three rule; it is not presented as a threshold tuned on a held-out test set.

V. CANDIDATE SET AND LABELING PROTOCOL

A. Candidate-Finding Unit

The evaluation unit is one LLM-proposed finding record, called a candidate finding in the paper. It is not an APK, class, line, raw keyword hit, or ecosystem prevalence estimate. A candidate finding represents one stable security-relevant claim identified by APK, entrypoint or class, sink/source behavior, category, and security consequence.

Candidate findings are merged when they refer to the same APK, same reachable entrypoint, same sink/source behavior, and same security consequence. Candidate findings are split when the same APK contains independently reachable entrypoints, when two sink/source behaviors create different

TABLE II
CANDIDATE SET COMPOSITION

Source	Candidates	Ref.+	Ref.-
PleOS-customized APKs	30	23	7
OWASP MASTG samples	4	4	0
InsecureBankv2	9	9	0
AOSP-derived samples	4	2	2
Total	47	38	9

security consequences, or when one candidate is blocked by a permission gate while another is blocked by route binding.

B. Reference Labels and Label Bias Boundary

Reference labels are reportability labels under the stated threat model. A reference-positive vulnerability finding is a candidate whose evidence package supports reporting the finding statically under the stated boundary. A reference-negative candidate is blocked by Android context or has no remaining security-relevant boundary.

External vulnerable corpora provide positive controls when the candidate maps to known vulnerable behavior. PleOS-customized findings are project-derived reportability labels based on manifest evidence, decompiled-code evidence, Android platform constraints, contextual verification notes, and public-safe report artifacts. AOSP-derived samples provide framework-blocking and false-positive controls. The label as-

TABLE III
TERMINOLOGY USED IN THE EVALUATION

Term	Meaning
LLM-proposed candidate findings	47 reviewable security claims produced by candidate generation.
Reference-positive candidates	38 candidates labelled reportable under the project threat model.
Final reported findings	37 candidates reported by the selected two-or-more-role rule.
Rejected false positives	9 LLM-proposed candidates rejected after context validation and review.
False negative	1 low-severity reference-positive candidate not reported by the final rule.

signment protocol is: create a candidate evidence package, apply Android context support/block checks, assign a reference-positive label when the claim remains reportable under the threat model, assign a reference-negative label when platform or route evidence blocks the claim, and record whether the label comes from a known vulnerable corpus or a PleOS/AOSP project-derived judgment.

These labels should not be treated as independent human labels. The PleOS-customized labels are intentionally project-derived and use some of the same Android context evidence that the workflow packages for reportability review. Therefore, the evaluation is best read as an audit of candidate-level adjudication under project reference labels, not as a fully independent human-label validation study. Independent semi-blind analyst review, trigger-only baseline measurement, and threshold-stability analysis remain future validation steps before making a peer-review-style generalization claim.

VI. CONTEXT VALIDATION AND ROLE-THRESHOLD ADJUDICATION

A. Evidence Package Schema

Each candidate is carried forward as an evidence package. The package contains the candidate claim, static evidence, Android context evidence, blocking controls, role-review decision, and claim boundary.

B. Android Context Checks

The context-validation layer answers claim-shaped questions. It asks whether a suspected provider exposure is actually exported and queryable, whether a suspected permission-abuse path has an external caller, whether a suspected receiver issue survives protected-broadcast semantics, and whether a sensitive value crosses a process, component, or trust boundary. Without a candidate claim, these checks would become a broader APK-wide rule-only discovery analyzer; that is a different baseline and is not the method evaluated here.

These checks are grounded in official Android and AOSP semantics rather than free-form LLM judgment. Component exposure depends on `android:exported` and Android 12 explicit-export requirements; provider exposure depends on `exported` state plus provider-level and read/write

TABLE IV
EVIDENCE PACKAGE FIELDS

Field	Purpose
Candidate claim	States the suspected security issue in reviewable form.
Static evidence	Names the APK, component, endpoint, sink/source behavior, category, severity, and public-safe evidence breadcrumb.
Android context evidence	Records manifest exposure, caller reachability, permission gates, route binding, protected-broadcast semantics, and framework controls.
Blocking controls	Explains why a suspicious code pattern does not survive as a reportable finding.
Role-threshold decision	Records which role-conditioned review lenses classify the claim as reportable.
Claim boundary	Separates static reportability from runtime reachability, data effect, UI effect, or unsupported vehicle-control claims.

permissions; signature permissions exclude ordinary same-device callers; protected broadcasts constrain receiver-trigger claims; and AAOS vehicle-adjacent claims must respect `CarProperty/VHAL` permission boundaries rather than inferring vehicle-control capability from APK-local code alone [13], [14], [16], [17], [19], [23], [44].

C. Role-Conditioned Review Lenses

The roles are not independent human experts and not different model vendors. They are three calibrated role-specific review prompts applied to the same evidence package. The threshold is therefore role-conditioned adjudication, not agreement among independent evaluators.

The attacker lens emphasizes abuse feasibility under the same-device app model, the defender lens emphasizes evidence sufficiency and fixability, and the IVI-domain lens checks whether the claim is meaningful without overstating vehicle-control impact. A candidate rejected by Android context validation may still be seen by the lenses as a hardening note, but it should not be counted as a final reported finding unless the evidence package supports reportability. This separation is important: context validation supplies the survival/blocking rationale, while the role-threshold rule decides final inclusion.

VII. EVALUATION DESIGN

A. Research Questions

The evaluation is organized around four research questions:

- **RQ1:** Does the broad LLM-generated candidate pool provide high-coverage input for candidate-level reportability adjudication within the constructed corpus?
- **RQ2:** Does Android context validation combined with a role-threshold adjudication rule reduce false-positive reports while retaining reference-positive claims?

TABLE V
EXAMPLES OF ANDROID CONTEXT SUPPORT AND BLOCKING RULES

Context Question	Supports Reportability When	Blocks Reportability When
Provider exposure	Provider is exported or queryable by a same-device app, no blocking permission is found, and returned rows cross a sensitive data boundary [17].	Provider is not externally reachable, is permission or signature-permission gated, or returns no security-relevant caller-observable data.
Receiver reachability	An untrusted same-device app can send the relevant action and the receiver path reaches security-sensitive behavior [13], [18].	The action is a protected broadcast such as <code>BOOT_COMPLETED</code> , or the receiver is effectively limited to framework/internal senders [19].
Permission-management path	The candidate includes caller-controlled package, account, route, or component data that can influence permission state.	Package or caller identity is internally route-bound, or framework permission checks prevent unprivileged influence [15], [16].
Component or service boundary	The endpoint is exported, implicitly reachable, or lacks an effective permission gate [13], [14].	The candidate targets explicit PleOS-internal components, requires signature/privileged permissions, or remains inside a trusted route.
Configuration or network claim	The configuration is active or tied to a reachable service path.	Evidence shows dead/test-only configuration or no reachable path under the stated threat model.

TABLE VI
ROLE-CONDITIONED REVIEW LENSES

Review Lens	Main Question	Typical Reject Reason
Attacker	Can an untrusted same-device caller plausibly abuse this?	No reachable caller, protected broadcast, or framework permission gate.
Defender	Is the evidence strong enough to report and fix?	Only a hardening concern or weak evidence package.
IVI-domain	Does this matter under PleOS/AAOS IVI trust boundaries?	Unsupported escalation into vehicle-control or physical vehicle-impact claims.

- **RQ3:** Which Android context conditions are responsible for rejecting false positives?
- **RQ4:** How do final reported findings distribute across APKs, severity, and AAOS/MASVS/TARA-oriented control areas?

B. Metrics and Statistical Test

We report precision, recall, and F1 as candidate-level adjudication metrics:

$$Precision = \frac{TP}{TP + FP}, \quad (1)$$

$$Recall = \frac{TP}{TP + FN}, \quad (2)$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}. \quad (3)$$

We use a paired McNemar exact test to compare a report-all candidate baseline and the final filtering rule over the same 47 candidates [45]. Because the role threshold is selected and evaluated on the same small corpus, the statistical result is exploratory evidence of candidate-level false-positive reduction, not a confirmatory generalization claim. Supporting bootstrap intervals use standard resampling to express small-sample uncertainty [46]. For exact binomial uncertainty on the final rule, the observed precision is $37/37 = 100.0\%$ with a two-sided 95% Clopper-Pearson lower bound of approximately 90.5%,

and the reference-negative rejection rate is $9/9 = 100.0\%$ with a lower bound of approximately 66.4%.

The evaluation does not measure APK-level discovery recall. The 47-candidate corpus combines PleOS-customized target APK candidates with external Android positive and negative controls. Pooled metrics are used to evaluate candidate-level adjudication behavior, not AAOS ecosystem prevalence or APK-wide discovery performance. The external controls stress false-positive rejection and known-vulnerable behavior; they should not be interpreted as production AAOS/IVI prevalence samples.

VIII. QUANTITATIVE RESULTS

A. Overall Candidate-Level Reportability Performance

The reported metrics compare candidate-level reportability decisions under the project-derived reference labels. Android context validation packages evidence and records why a claim survives or is blocked; role-threshold adjudication then decides whether the evidence package is included in the final reported set. Because context validation is not reported as a standalone classifier in this archive, the metric table compares the report-all candidate baseline with role-threshold decision rules over the same constructed candidate corpus. This is not a comparison against a full APK-wide rule-only discovery scanner.

The LLM report-all candidate baseline contains all 38 reference-positive candidates in the constructed candidate universe, but also includes 9 reference-negative candidates. This

TABLE VII
MAIN CANDIDATE-LEVEL EVALUATION

Evaluation View	Rep.	TP	FP	FN	P	R	F1
LLM report-all candidate baseline	47	38	9	0	80.9%	100.0%	0.894
≥ 1 role-threshold rule	45	38	7	0	84.4%	100.0%	0.916
Final ≥ 2 role-threshold rule	37	37	0	1	100.0%	97.4%	0.987
3/3 role-threshold rule	33	33	0	5	100.0%	86.8%	0.930

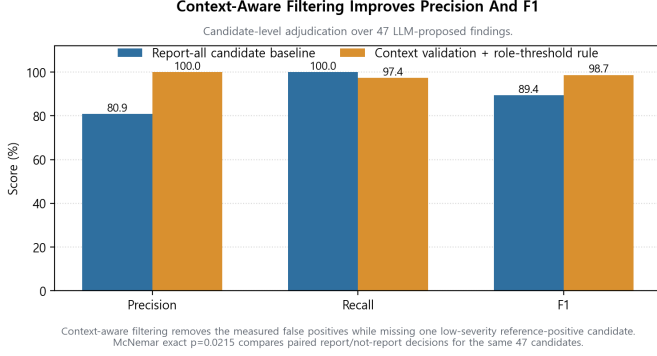


Fig. 2. Candidate-level precision, recall, and F1 for the LLM report-all candidate baseline and final context-aware filtering rule on the 47-candidate reference set.

means the broad candidate-generation step is useful as a high-coverage candidate pool inside the constructed corpus, not evidence of APK-level discovery recall. The selected final rule reports 37 findings with zero false positives observed among the 9 reference-negative candidates in this 47-candidate corpus and 1 low-severity false negative. Its F1 score is 0.987. Because the final threshold was selected and evaluated on the same small corpus, the row should be read as an archive measurement of this corpus rather than a held-out estimate of future precision.

The negative evidence base is small: only 9 of the 47 candidates are reference-negative, and the external vulnerable corpora are intentionally positive-heavy. This composition is useful for preserving known-vulnerable positive controls, but it limits any strong claim about false-positive robustness. A trigger-only keyword/rule baseline, a manifest-rule baseline, and a context-validation-only ablation would be useful for future peer-review validation before comparing this workflow against conventional static-analysis baselines.

B. Role-Threshold Trade-Off

Figure 3 shows the role-threshold trade-off. Requiring one role lens to classify a claim as reportable preserves candidate-level recall but leaves seven false positives. Requiring all three role lenses preserves precision but misses five reference-positive candidates. The selected two-of-three role-threshold rule reports 37 findings with zero false positives observed among the 9 reference-negative candidates and one low-severity false negative.

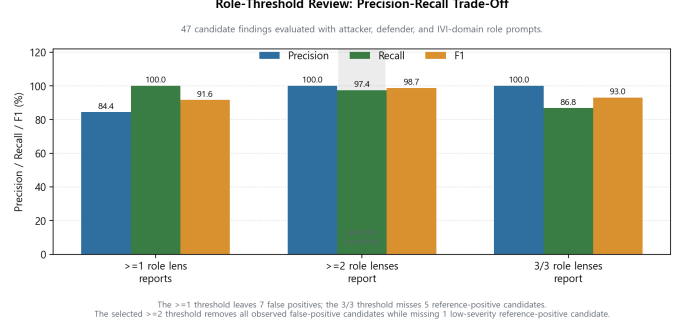


Fig. 3. Precision-recall trade-off for role-threshold adjudication on the 47-candidate corpus. The selected threshold removes all observed false-positive candidates in this reference set while missing one low-severity reference-positive candidate.

C. False-Negative Case: $\forall C-7$

The one false negative is $\forall C-7$, a low-severity VehicleControl candidate involving receiver-registration flag suppression. It is reference-positive as a hardening-style reportability label, but only the defender lens treated it as reportable, while the attacker and IVI-domain lenses did not find enough abuse feasibility or IVI trust-boundary impact to meet the final reporting threshold. This case illustrates the main trade-off of the conservative two-of-three rule: it removes the observed false-positive candidates in this corpus, but can drop low-severity hardening issues whose evidence is useful to defenders yet weak as an abuse claim.

D. Category-Level Outcome Distribution

Figure 4 separates retained and rejected candidates by vulnerability category. The 37 retained findings concentrate in intent/component exposure (13), hardcoded-secret or logging-style evidence (11), network or transport claims (7), cryptographic misuse (5), and one reflection/dynamic-loading claim. The rejected candidates concentrate in intent (5), permission-management (3), and network (2) categories, which is consistent with the workflow's purpose: Android context checks are most important when a suspicious pattern may be blocked by route binding, permission gates, protected broadcasts, or missing reachability.

E. Paired Correctness Comparison

The paired McNemar table has one candidate correct only under the report-all candidate baseline and nine candidates corrected only by the final filtering rule. The exact two-tailed McNemar test gives $p = 0.0215$. Because the selected

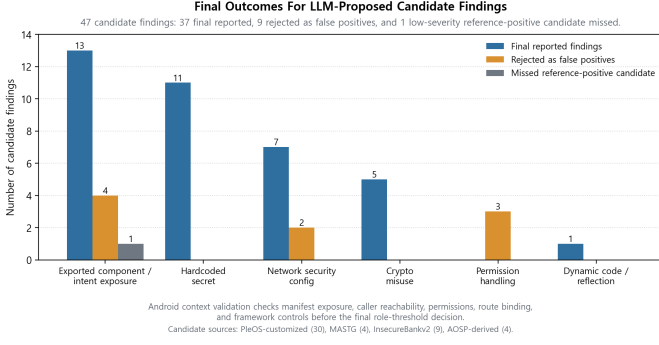


Fig. 4. Final outcomes for LLM-proposed candidate findings by vulnerability category.

threshold is evaluated on the same small corpus used to choose it, this result is exploratory evidence of candidate-level false-positive reduction.

IX. QUALITATIVE CASE ANALYSIS

A. Context Conditions Behind Filtering

The nine false positives were not random errors. They were suspicious static patterns whose exploitability depended on Android context. WebView file/network patterns lacked an external producer, permission-management APIs used internally route-bound package-name sources, a USB permission path was blocked by framework checks, a boot receiver was blocked by protected-broadcast semantics, one candidate preserved user-mediated AppMarket navigation rather than an auto-install primitive, another used explicit PleOS-internal component targeting, and a voice-service candidate was gated by a signature-level voice-interaction permission. These cases show that context validation defines whether a suspicious pattern remains a reportable claim under the stated same-device threat model.

B. *Imp-1: Surviving Provider Exposure Claim*

Initial suspicious signal: manifest/provider evidence identifies a prompt-provider surface associated with IVI-local LLM prompt or corpus data. **Candidate claim:** a same-device unprivileged Android app may query sensitive provider rows. **Static evidence:** the evidence package records the provider/component breadcrumb and public-safe manifest or decompiled-code evidence rather than publishing raw PleOS snippets. **Android context check:** the claim survives because the package does not identify a signature permission, protected framework route, or caller binding that blocks the same-device query path; local emulator query evidence may strengthen reachability when response rows are redacted. **Decision:** retain as a final reported finding. **Claim boundary:** the public artifact reports same-device provider exposure under emulator/static evidence boundaries; it does not claim remote exploitation, LLM service compromise, public prompt dumping, or vehicle actuation.

C. *ssl-6 / SysLogService: Surviving gRPC Plaintext Transport Claim*

Initial suspicious signal: decompiled/config evidence indicates plaintext or weakly protected transport for a diagnostic, logging, or service communication path. **Candidate claim:** sensitive IVI-local service data may cross a network or communication boundary without the expected transport protection. **Android context check:** the claim remains reportable when the evidence package preserves the relevant communication path and does not reduce the issue to a dead configuration entry or unreachable test-only path. **Decision:** retain as a final reported finding. **Claim boundary:** the finding is a static transport-security reportability claim, not proof of remote compromise or complete traffic interception in a production vehicle.

D. *vc-3/4: Route-Bound Permission Path Rejection*

Initial suspicious signal: permission-management APIs such as grant or revoke calls appear security-sensitive in the decompiled code. **Candidate claim:** an untrusted same-device caller may influence permission state through the exposed path. **Android context check:** route and package-name analysis shows that the relevant package or caller route is internally bound rather than attacker-controlled. **Decision:** reject as a false-positive candidate under the stated threat model. **Claim boundary:** the code may remain useful for hardening review, but it is not a reportable same-device permission-abuse finding without caller influence.

E. *ss-1: Protected Broadcast False Positive*

Initial suspicious signal: an exported boot-related receiver appears reachable in manifest or decompiled-code evidence. **Candidate claim:** an untrusted same-device app may trigger initialization behavior by sending the broadcast. **Static evidence:** the receiver/action pair is a plausible trigger surface before Android platform semantics are applied. **Android context check:** the relevant action is `BOOT_COMPLETED`, which AOSP declares as a protected broadcast; ordinary apps cannot emit it as the triggering sender under the stated model. **Decision:** reject as a false-positive candidate because the attacker lacks the triggering capability. **Claim boundary:** the receiver may still be relevant for defensive hardening, but it is not reportable as an externally triggerable receiver issue under this threat model.

F. Reported Findings by APK and Severity

High-severity final reported findings concentrate in credential, logging, network, and component surfaces rather than being uniformly distributed across APKs. This distribution guides case prioritization and runtime evidence selection.

X. AUTOMOTIVE SECURITY MAPPING

The mapping layer translates candidate-level static findings into automotive-security reporting language. The 37 final reported findings are assigned to AAOS security areas,

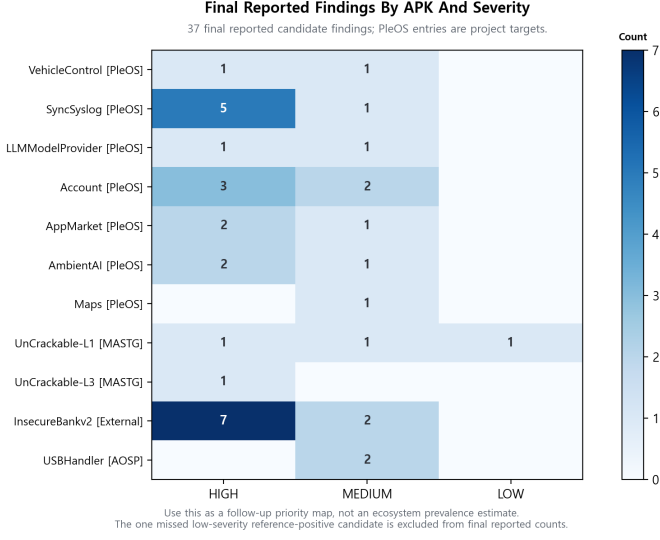


Fig. 5. Final reported findings by APK and severity inside the constructed candidate corpus. The figure is a follow-up priority map, not an ecosystem prevalence estimate; InsecureBankv2 is an intentionally vulnerable positive-control corpus.

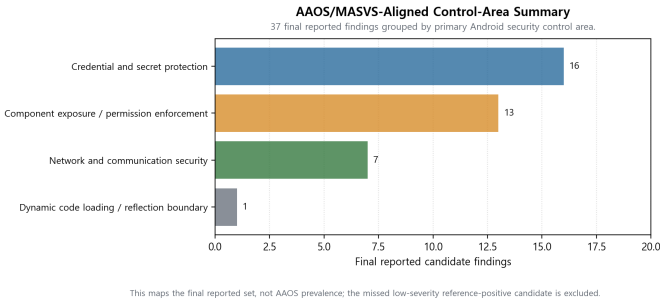


Fig. 6. AAOS/MASVS-aligned control-area summary for the 37 final reported findings.

OWASP MASVS-oriented categories, and TARA-oriented asset, threat, risk-level, and treatment language. Provider exposure is treated as an IVI-local data-boundary issue aligned with IPC/component-exposure testing vocabulary, while credential, logging, and transport findings are translated into secret-handling and communication-security control areas. This mapping is downstream of reportability: it supports prioritization and communication for OEM, IVI, and security-review stakeholders, but it does not convert a static APK finding into a vehicle-safety or actuation claim.

XI. SUPPORTING VALIDATION AND RUNTIME BOUNDARY

Supporting validation tracks clarify uncertainty, robustness, blind spots, and boundaries around the main static evaluation. They are not additional end-to-end performance claims and do not replace the 47-candidate candidate-level metric; Appendix Table A2 separates the paired test, bootstrap uncertainty, retrieval consistency, native-code boundary scan, independent LLM cross-review, and runtime PoC boundary. The runtime PoC track may strengthen selected static observations into same-device reachability, local data/state effect, or emulator-

visible UI/IVI behavior, but it does not claim remote exploitation or physical vehicle control.

XII. DISCUSSION

A. Claim Scaffolds Need Android Semantics

The LLM-assisted candidate-generation step is useful because it converts triaged decompiled code into reviewable claim packages, but its role is not to finalize vulnerability decisions. The report-all candidate baseline acts as a high-coverage pool while retaining nine false positives; Android context validation then applies exported state, permission gates, route binding, protected broadcasts, and other platform semantics to decide whether each claim remains reportable. The method is therefore closer to LLM-assisted SAST alert or reportability adjudication than to an LLM vulnerability detector.

B. Cross-Review and Label Robustness

The independent LLM cross-review did not beat the calibrated role-threshold workflow, which supports the practical value of prompt calibration and evidence packaging. However, the same result can also be read as a label-robustness warning: a separate model-family consensus rejected 9 reference-positive candidates. Those disagreements should be adjudicated by human review before this archive is promoted into a peer-review-style benchmark claim.

C. Software-Defined Vehicle Security Positioning

This work is positioned within software-defined vehicle security because Android Automotive and IVI applications combine mobile-app attack surfaces with vehicle-adjacent trust boundaries. The contribution is not a general LLM benchmark, but a bounded review workflow for translating decompiled APK evidence into reportable automotive-security findings under a conservative same-device threat model.

D. Practical Use in IVI Security Review

The workflow is most useful as a review scaffold. It provides candidate claims, Android context evidence, role-threshold reportability decisions, and public-safe mapping artifacts. Human reviewers can audit the evidence package and claim boundary instead of reading raw LLM outputs in isolation. A production-grade comparison would still need trigger-only, manifest-rule, and tool-based baselines, plus independent label validation over the same candidate units.

XIII. LIMITATIONS AND FUTURE WORK

The main limitations are label bias, corpus selection, small sample size, incomplete Android reachability modeling, heuristic obfuscation screening, and incomplete native/runtime coverage. This evaluation does not measure APK-level discovery recall because the candidate universe is built from LLM-proposed findings and selected control samples. The role threshold is selected and evaluated on the same small corpus, so threshold-specific metrics may overstate generalization. PleOS-customized labels are project-derived and

should receive independent semi-blind analyst validation before stronger benchmark claims are made. The negative evidence base is also small, with 9 reference-negative candidates, so the result of zero false positives observed among those candidates should be read with the exact lower bounds reported above.

Future peer-review validation should add a trigger-only keyword/rule baseline, a manifest-rule baseline, a context-validation-only ablation, and feasible tool comparisons such as MobSF-style static checks, Android Lint/security checks, Semgrep or JADX-grep rules, and, where artifact constraints permit, FlowDroid/IccTA/Amandroid-style analyses. It should also report threshold stability through leave-one-out or bootstrap analysis, independent label agreement and disagreement categories, and additional production-like AAOS/IVI APKs. Retrieval-augmented prompting, deeper native/runtime analysis, and emulator PoC evidence should remain supporting tracks unless they are evaluated as separate end-to-end metrics.

XIV. CONCLUSION

This paper presents a reportability-centered LLM-assisted static security review workflow for Android Automotive IVI APKs. On a 47-candidate reportability corpus, the final two-of-three role-threshold rule retains 37 of 38 reference-positive candidates and reports zero false positives among the 9 reference-negative candidates. The result supports the central thesis: LLMs are most useful in this setting as claim-generation and review scaffold components, while Android context validation and conservative reportability boundaries remain the final authority.

APPENDIX A

SUPPLEMENTARY FIGURES AND VALIDATION ARTIFACTS

The appendix collects supporting views that help interpret the main candidate-level evaluation. These figures do not add a separate APK-wide benchmark, prevalence estimate, or runtime exploit claim. They explain the construction of the 47-candidate corpus, the uncertainty around the report-all candidate baseline, the decompiled-code readability screen used before review, and the supporting validation tracks that bound the main result.

A. Candidate Corpus Composition

Figure A1 explains where the 47 candidate findings came from. Each unit is one candidate finding, not one APK, one class, or one ecosystem prevalence sample. The PleOS-customized candidates form the IVI target set, while OWASP MASTG, InsecureBankv2, and AOSP-derived candidates provide external positive and negative controls for reportability adjudication.

Table A1 provides a source-stratified view to make clear that the pooled metric is not an AAOS prevalence estimate. The PleOS-customized subset preserves the same pattern as the pooled result—false-positive removal with one low-severity miss—while the external controls serve as positive and negative reportability controls rather than production IVI samples. In the external-control subset, the final rule reports

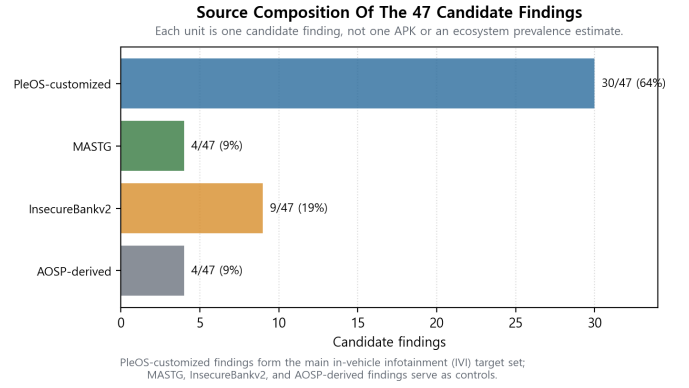


Fig. A1. Source composition of the 47 candidate findings.

15 reference-positive findings and rejects 2 reference-negative candidates.

B. Uncertainty Around the Report-All Candidate Baseline

Figure A2 is included because the corpus is small. It shows bootstrap uncertainty for the report-all candidate baseline, where all 47 LLM-generated candidates are treated as reported. The figure should be read as uncertainty around the baseline candidate pool, not as a universal false-positive or recall guarantee.

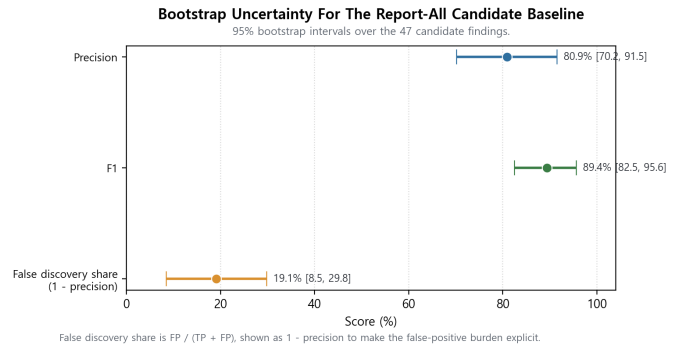


Fig. A2. Bootstrap uncertainty for the report-all candidate baseline.

C. Decompiler Readability and Identifier Obfuscation

Figures A3 and A4 document the preprocessing readability screen. The screen flags classes with high-obfuscation identifier-name patterns in JADX-decompiled output, such as synthetic class and method names. It is a review-planning heuristic: it describes how difficult decompiled code may be to inspect, not vulnerability severity, reportability, or final finding quality.

D. Supporting Validation Tracks

Table A2 separates measurement roles that should not be pooled into one score. The primary role-threshold result is the main candidate-level decision metric. The paired test, independent LLM cross-review, native-code boundary scan, and retrieval-grounded consistency check each answer a different robustness or boundary question around that main result.

TABLE A1
SOURCE-STRATIFIED CANDIDATE-LEVEL METRICS

Source View	Decision Rule	Rep.	TP	FP	FN	P	R	F1
PleOS-customized candidates	Report-all baseline	30	23	7	0	76.7%	100.0%	0.868
PleOS-customized candidates	Final ≥ 2 role rule	22	22	0	1	100.0%	95.7%	0.978
External Android controls	Final ≥ 2 role rule	15	15	0	0	100.0%	100.0%	1.000

TABLE A2
SUPPORTING VALIDATION TRACKS AND THEIR MEASUREMENT ROLES

Track	Question Answered	Measured Result and Interpretation
Primary role-threshold adjudication	Which candidate findings should be reported under the selected role-threshold rule?	Final two-of-three role-threshold rule: TP 37, FP 0, FN 1 against the 38-finding reference-positive set.
Paired comparison	Does the final filtering rule improve paired candidate-level correctness over the report-all candidate baseline?	McNemar exact test $p = 0.0215$ over the same 47 candidate findings.
Independent LLM cross-review	Does a separate model-family cross-review outperform the primary role-threshold adjudication rule?	Precision 100.0%, recall 76.3%, F1 0.866; it did not outperform the selected workflow and also marks 9 reference-positive candidates for later adjudication.
Native-code boundary scan	Did sampled native libraries add Java/Kotlin-missed native-code-boundary vulnerabilities?	4 .so samples checked; 0 additional native-code-boundary vulnerabilities found.
Retrieval-grounded consistency check	Does the local retrieval corpus preserve verdict and AAOS-category consistency?	Verdict agreement 85.1% and category agreement 85.1%; this is not an end-to-end LLM improvement claim.

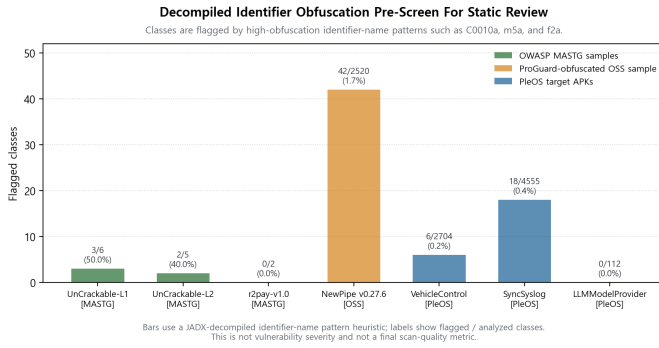


Fig. A3. Decompiled identifier obfuscation pre-screen for static review.

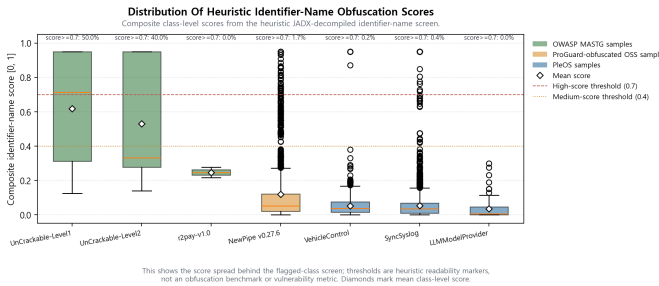


Fig. A4. Distribution of heuristic identifier-name obfuscation scores.

E. Baseline Design for Peer-Review Extension

Table A3 defines fair comparison units for future baseline work without claiming those measurements in this archive. The first group targets candidate discovery burden; the second group tests generic static-tool overlap; the third group is better

treated as feasibility or subset analysis because whole-app taint and ICC tools optimize a different objective than candidate-level reportability adjudication.

APPENDIX B ARTIFACT INDEX AND REDACTION POLICY

This appendix section records the publication boundary for companion artifacts. The public artifact set includes redacted reports, aggregate metrics, public-safe figures, label schemas, deterministic scripts, prompt templates, role-lens rubrics, and the public-safe evidence package table. Raw APKs, JADX output, prompt rows, secrets, logs, videos, unredacted runtime evidence, local Stage 3 outputs, and generated harness outputs remain local-only. This boundary is required because PleOS/OEM code excerpts and sensitive runtime evidence cannot be published as-is.

Table B1 lists the companion artifacts that make the archive auditable without exposing proprietary or sensitive material.

ACKNOWLEDGMENT

Generative AI tools were used during manuscript preparation for draft organization, language editing, and rewriting of non-final explanatory text. They were not used as autonomous sources for security labels, candidate-level metrics, reference labels, or final vulnerability conclusions. The author remains responsible for checking all technical claims, labels, metrics, references, and reportability decisions.

TABLE A3
BASELINE DESIGN FOR PEER-REVIEW EXTENSION

Baseline	Feasibility	Fair Comparison Unit	Expected Limitation
Trigger-only keyword/rule	High	Candidate generation recall	High recall but little Android context.
Android manifest rule	High	Component exposure candidate precision	Weak code-level consequence evidence.
MobSF-style static checks	Medium-high	Finding category overlap	Generic mobile findings may miss OEM IVI semantics.
Android Lint/security checks	Medium	Source-available subset only	Source/build-context assumptions do not fit all APK artifacts.
Semgrep/JADX grep	High	Syntactic candidate hit rate	Route, permission, and protected-broadcast semantics remain weak.
FlowDroid/IccTA/Amandroid	Medium	Flow or ICC subset feasibility	Different objective from reportability adjudication; timeout/setup risk.

TABLE B1
PUBLIC-SAFE COMPANION ARTIFACTS

Artifact	Purpose
Evidence package table	Public-safe evidence package summary for candidate findings.
Prompt templates and role rubrics	Recreate the review protocol without exposing private prompt rows.
Aggregate reports and figures	Reproduce headline tables, charts, and mapped reporting artifacts.
Redacted public reports	Inspect public-safe case narratives and claim boundaries.

TABLE B2
PUBLIC-ONLY VERSUS LOCAL-EVIDENCE REPRODUCIBILITY

Workflow	Reproducibility Boundary
Public-only inspection	Readers can inspect redacted reports, aggregate metrics, public-safe figures, label schemas, deterministic scripts, and the evidence-package table.
Full row-level recomputation	Requires local/private reports, unredacted evidence, raw JADX output, and local Stage 3 role outputs that cannot all be published safely.
Future validation	Independent human review sheets and trigger-only baseline worksheets can be published once completed, but are not claimed as measured results in this archive.

REFERENCES

- [1] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. L. Traon, D. Ocateau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2014.
- [2] F. Wei, S. Roy, X. Ou, and Robby, "Amandroid: A precise and general inter-component data flow analysis framework for security vetting of Android apps," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2014.
- [3] L. Li, A. Bartel, T. F. Bissyandé, J. Klein, Y. L. Traon, S. Arzt, S. Rasthofer, E. Bodden, and D. Ocateau, "IccTA: Detecting inter-component privacy leaks in Android apps," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2015.
- [4] M. I. Gordon, D. Kim, J. Perkins, L. Gilham, N. Nguyen, and M. Rinard, "Information-flow analysis of Android applications in DroidSafe," in *Proceedings of the Network and Distributed System Security Symposium*, 2015.
- [5] L. Lu, Z. Li, Z. Wu, W. Lee, and G. Jiang, "CHEX: Statically vetting Android apps for component hijacking vulnerabilities," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2012.
- [6] F. Pauck, E. Bodden, and H. Wehrheim, "Do Android taint analysis tools keep their promises?" in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM Press, 2018.
- [7] E. Chin, A. P. Felt, K. Greenwood, and D. Wagner, "Analyzing Inter-Application communication in Android," in *Proceedings of the 9th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 2011, pp. 239–252.
- [8] D. Ocateau, P. McDaniel, S. Jha, A. Bartel, E. Bodden, J. Klein, and Y. L. Traon, "Effective Inter-Component communication mapping in Android: An essential step towards holistic security analysis," in *22nd USENIX Security Symposium*. USENIX Association, 2013, pp. 543–558. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/ocateau>
- [9] J. Samhi, A. Bartel, T. F. Bissyandé, and J. Klein, "RAICC: Revealing atypical inter-component communication in Android apps," in *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*, 2021. [Online]. Available: <https://arxiv.org/abs/2012.09916>
- [10] Y. K. Lee, J. Y. Bang, G. Safi, A. Shahbazian, Y. Zhao, and N. Medvidovic, "A SEALANT for inter-app security holes in Android," in *Proceedings of the IEEE/ACM 39th International Conference on Software Engineering*, 2017, pp. 312–323.
- [11] B. F. Demissie, M. Ceccato, M. Röser, and A. Türpe, "Identifying Android inter app communication vulnerabilities using static and dynamic analysis," in *Proceedings of the 3rd International Conference on Mobile Software Engineering and Systems*, 2016, pp. 255–266.
- [12] J. Garcia, M. Hammad, N. Ghorbani, and S. Malek, "Automatic generation of inter-component communication exploits for Android applications," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 661–671.
- [13] Android Developers, "android:exported," <https://developer.android.com/privacy-and-security/risks/android-exported>, accessed: May 20, 2026.
- [14] —, "Behavior changes: Apps targeting Android 12," <https://developer.android.com/about/versions/12/behavior-changes-12>, accessed: May 20, 2026.
- [15] —, "Permission-based access control to exported components," <https://developer.android.com/privacy-and-security/risks/access-control-to-exported-components>, accessed: May 20, 2026.
- [16] —, "Permissions on android," <https://developer.android.com/guide/topics/permissions/overview>, accessed: May 20, 2026.
- [17] —, "Provider manifest element," <https://developer.android.com/guide/topics/manifest/provider-element>, accessed: May 20, 2026.
- [18] —, "Insecure broadcast receivers," <https://developer.android.com/privacy-and-security/risks/insecure-broadcast-receiver>, accessed: May 20, 2026.

- [19] Android Open Source Project, “AndroidManifest.xml protected broadcast declarations,” https://android.googlesource.com/platform/frameworks/base/-/android-14.0.0_r1/core/res/AndroidManifest.xml, accessed: May 20, 2026.
- [20] Android Developers, “Android for cars overview,” <https://developer.android.com/training/cars>, accessed: May 20, 2026.
- [21] —, “Android automotive os overview,” <https://developer.android.com/training/cars/platforms/automotive-os>, accessed: May 20, 2026.
- [22] Android Open Source Project, “Vehicle system isolation,” https://source.android.com/docs/automotive/security/vehicle_system_isolation, accessed: May 20, 2026.
- [23] —, “VHAL overview,” <https://source.android.com/docs/automotive/vhal>, accessed: May 20, 2026.
- [24] S. Mazloom, M. Rezaeirad, A. Hunter, and D. McCoy, “A security analysis of an In-Vehicle infotainment and app platform,” in *10th USENIX Workshop on Offensive Technologies (WOOT 16)*. Austin, TX: USENIX Association, Aug. 2016.
- [25] A. Mandal and A. Cortesi, “Static analysis of Android Auto infotainment and on-board diagnostics II apps,” *Software: Practice and Experience*, 2019.
- [26] M. Pese, K. Shin, J. Bruner, and A. Chu, “Security analysis of Android Automotive,” *SAE International Journal of Advances and Current Practices in Mobility*, vol. 2, no. 4, pp. 2337–2346, 2020.
- [27] J. Manana, P. Vyas, and Y. Aafer, “Red light for security: Uncovering auto feature check and access control gaps in AAOS,” in *Detection of Intrusions and Malware, and Vulnerability Assessment*, ser. Lecture Notes in Computer Science, vol. 15748. Springer, 2025, pp. 147–166.
- [28] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proceedings of the 13th International Conference on Learning Representations*, 2025.
- [29] H. Li, Y. Hao, Y. Zhai, and Z. Qian, “Assisting static analysis with large language models: A ChatGPT experiment,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 2107–2111.
- [30] W. Klieber and L. Flynn, “Using LLMs to adjudicate static-analysis alerts,” in *Proceedings of the 58th Hawaii International Conference on System Sciences*, 2025, pp. 7556–7564.
- [31] M. M. Mohajer, R. Aleithan, N. S. Harzevili, M. Wei, A. B. Belle, H. V. Pham, and S. Wang, “SkipAnalyzer: A tool for static code analysis with large language models,” arXiv:2310.18532, 2023. [Online]. Available: <https://arxiv.org/abs/2310.18532>
- [32] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, “LLMs in software security: A survey of vulnerability detection techniques and insights,” *ACM Computing Surveys*, 2025.
- [33] T. Zhang, C. Yang, Y. Su, M. Weyssow, H. Nguyen, T. Bui, H. J. Kang, Y. Li, E. L. Ouh, L. K. Shar, and D. Lo, “Benchmarking large language models for multi-language software vulnerability detection,” arXiv:2503.01449, 2025. [Online]. Available: <https://arxiv.org/abs/2503.01449>
- [34] OWASP, “OWASP mobile application security verification standard (MASVS),” <https://mas.owasp.org/MASVS/>, accessed: May 20, 2026.
- [35] —, “OWASP mobile application security testing guide (MASTG),” <https://mas.owasp.org/MASTG/>, accessed: May 20, 2026.
- [36] —, “Mobile application security weakness enumeration (MASWE),” <https://mas.owasp.org/MASWE/>, accessed: May 20, 2026.
- [37] —, “MASTG-TEST-0029: Testing for sensitive functionality exposure through IPC,” <https://mas.owasp.org/MASTG-TEST-0029/>, accessed: May 20, 2026.
- [38] —, “MASTG-KNOW-0017: App permissions,” <https://mas.owasp.org/MASTG-KNOW-0017/>, accessed: May 20, 2026.
- [39] UNECE, “Un regulation no. 155 – cyber security and cyber security management system,” <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security>, 2021, accessed: May 20, 2026.
- [40] ISO, “ISO/SAE 21434:2021 – road vehicles – cybersecurity engineering,” <https://www.iso.org/standard/70918.html>, 2021, accessed: May 20, 2026.
- [41] HEAVENS Consortium, “Deliverable D2: Security models,” https://autosec.se/wp-content/uploads/2018/03/HEAVENS_D2_v2.0.pdf, 2016, release 2, Version 2.0.
- [42] A. Lautenbach, M. Almgren, and T. Olovsson, “Proposing HEAVENS 2.0 – an automotive risk assessment model,” in *Proceedings of the ACM Computer Science in Cars Symposium*, 2021, pp. 1–16.
- [43] O. Henniger, A. Ruddle, H. Seudié, B. Weyl, M. Wolf, and T. Wollinger, “Securing vehicular on-board IT systems: The EVITA project,” in *Proceedings of the 25th VDI/VW Automotive Security Conference*, Ingolstadt, Germany, 2009.
- [44] Android Developers, “VehiclePropertyIds,” <https://developer.android.com/reference/android/car/VehiclePropertyIds>, accessed: May 20, 2026.
- [45] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, 1947.
- [46] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, 1979.